

LU2IN006 - Simulateur de CPU

—

par Kyrylo Hoviadin et Victor Ji - Groupe 10 n°13

Sommaire

Objectif principal

Structure du projet

Implémentations des composants

Fonctionnalités clés

Tests et vérifications

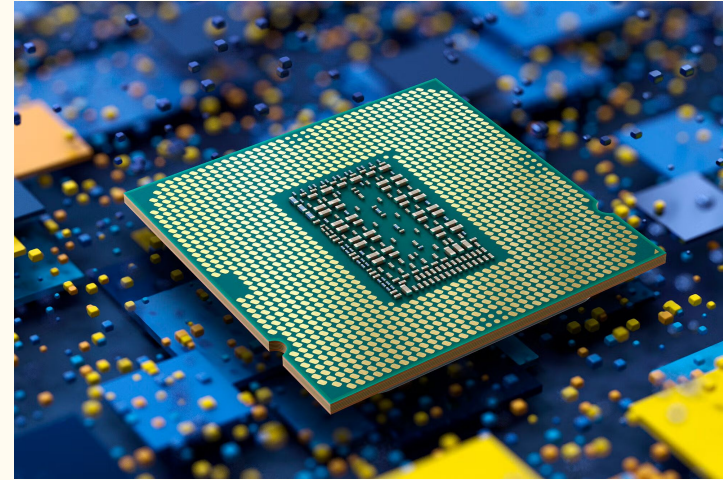
Performances et optimisations

Difficultés rencontrées

Conclusion

Objectif principal

- Identifier les composants clés d'un CPU:
 - Gestion de registres (AX, BX, CX, DX)
 - Mémoire segmentée (données, code, pile)
 - Implémentation de différents modes d'adressage
 - Cycle d'instruction complet
- Réalisation d'un simulateur de CPU simplifié en langage C
- Modélisation du fonctionnement d'un processeur exécutant des programmes en pseudo-assembleur



Architecture du projet

- Le projet se compose en 3 dossiers principaux:
 - include : fichiers headers (.h)
 - src : fichiers sources (.c)
 - tests : tests unitaires (.c)
- Elle contient également les fichiers complémentaires suivants:
 - README : description détaillée de la réalisation du projet.
 - Makefile : exécuter proprement le projet et tester les différentes parties de notre CPU.
 - exemple.asm : code pseudo-assembleur pour tester la fonctionnalité du processeur.

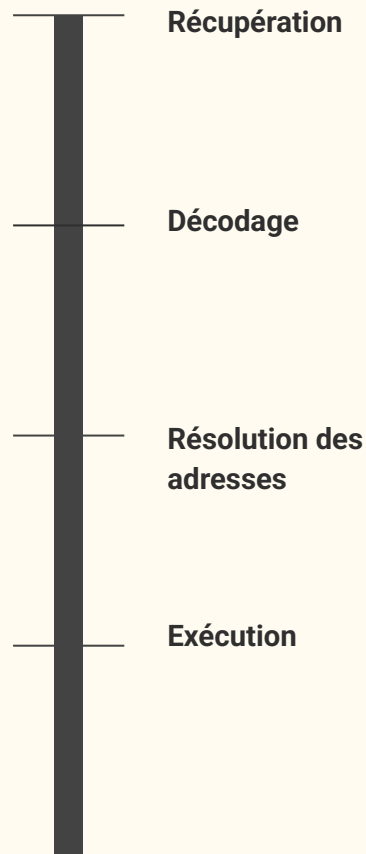
```
▼ 13-HOVIADIN-JI
  ▼ include
    C addressing.h
    C cpu.h
    C hashmap.h
    C instructions.h
    C memory_handler.h
    C parser.h
  ▼ src
    C addressing.c
    C cpu.c
    C hashmap.c
    C instructions.c
    C main.c
    C memory_handler.c
    C parser.c
  ▼ tests
    C test_addressing.c
    C test_cpu.c
    C test_hashmap.c
    C test_memory.c
    C test_parser.c
  ≡ cpu_simulator.exe
  ASM exemple.asm
  iterations_perf.csv
  ≡ main
  M makefile
  ≡ plan_detailee.txt
  ⓘ README.md
```

Implémentation des composants

- **hashmap.c** : Table de hachage pour gérer les associations clé-valeur.
- **memory_handler.c** : Module de gestion mémoire avec fusion de blocs libres et stratégies d'allocation (First/Best/Worst Fit).
- **parser.c** : Conversion du code pseudo-assembleur (.DATA/.CODE) en structures exploitables par le CPU.
- **cpu.c** : Coordination de l'exécution des instructions, gestion des registres et interaction avec la mémoire (Processus central).
- **addressing.c** : Résolution automatique des différents types d'adressage.
- **instructions.c** : Implémentation du jeu d'instructions.
- **main.c** : Implémentation du coeur central de notre simulateur.

Fonctionnalités clés

- **Cycle d'instruction complet:** fetch-decode-execute
- **Modes d'adressage:**
 - Immédiat (valeurs littérales: 42)
 - Registre (AX, BX, CX, DX)
 - Direct mémoire ([5])
 - Indirect via registre ([AX])
- **Jeu d'instructions:**
 - Transfert : MOV, PUSH, POP
 - Arithmétique : ADD, CMP
 - Saut, arrête : JMP, JZ, JNZ, HALT
- **Gestion de la mémoire:** Segments données (DS), code (CS), pile (SS) et extra (ES)

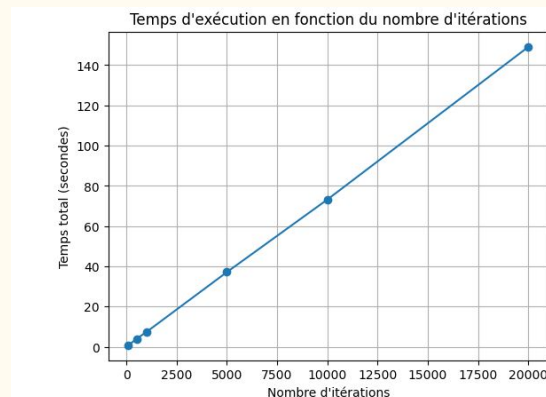


Tests et vérifications

- Tests unitaires organisés par module fonctionnel:
 - **test_addressing.c**: Validation des quatre modes d'adressage
 - **test_cpu.c**: Mesure des temps d'initialisation et destruction
 - **test_hashmap.c**: Vérification des opérations principales (insertion, lecture, suppression)
 - **test_memory.c**: Test des fonctions du gestionnaire de mémoire
 - **test_parser.c**: Validation de la conversion du code assembleur
- Tests Valgrind confirment l'absence de fuites mémoire critiques.

Performances et optimisations

- Hashmap: Implémentation simple avec possibilité d'optimisation (sondage, brassage des clés)
- Parser: Complexité $O(n)$ avec progression linéaire
- CPU: Performances linéaires $O(n)$ pour l'exécution des instructions
- Temps négligeable pour des programmes simples $O(1)$
- Optimisations possibles:
 - Redimensionnement dynamique de la table de hachage
 - Amélioration de la sensibilité du parser



Difficultés rencontrées

- Sensibilité du parser: espaces, sauts de ligne, tabulations
- Résolution des labels et des adresses mémoire
- Cohérence entre les différents modules du simulateur
- Mauvaise organisation sur les tâches à effectuer

```
src/parser.c:42:21: warning: unused variable 'remain_len' [-Wunused-variable]
  42 |         int remain_len = strlen(substr + key_len);
      |         ~~~~~^~~~~~
make: *** [makefile:18: tests/test_addressing] Error 1
vijib@MB14:/mnt/c/Users/vijib/Downloads/prooojet123/prooojet123$
Test environment initialized.
Résultats des tests d'adressage :
Immediat (42) => 42
Registre (AX) => 3
Direct ([5]) => 55
Indirect ([AX]) => 35
Erreur free_es_segment: segment ES non trouve
vijib@MB14:/mnt/c/Users/vijib/Downloads/prooojet1234/Kyryloo$
```



Conclusion

- Simulateur CPU basique mais solide
- Performances des implémentations en $O(n)$
- Pas de fuites mémoire ni d'erreurs de syntaxe (d'après Valgrind)
- Possibilités d'amélioration:
 - Optimisation de la libération des octets alloués
 - Redimensionnement dynamique de la table de hachage
 - Extension du jeu d'instructions
- Projet permettant une compréhension approfondie du fonctionnement d'un CPU

```
==1028== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==1028== Command: make
==1028==
make: Nothing to be done for 'all'.
==1028==
==1028== HEAP SUMMARY:
==1028==   in use at exit: 144,136 bytes in 1,226 blocks
==1028==   total heap usage: 2,182 allocs, 956 frees, 472,671 bytes allocated
==1028==
==1028== LEAK SUMMARY:
==1028==   definitely lost: 0 bytes in 0 blocks
==1028==   indirectly lost: 0 bytes in 0 blocks
==1028==   possibly lost: 0 bytes in 0 blocks
==1028==   still reachable: 144,136 bytes in 1,226 blocks
==1028==   suppressed: 0 bytes in 0 blocks
==1028== Rerun with --leak-check=full to see details of leaked memory
==1028==
==1028== For lists of detected and suppressed errors, rerun with: -s
==1028== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
vijibo@MB14:/mnt/c/Users/vijib/Downloads/prooojet1234/13-HOVIADIN-DI$
```

